hashlib — Resumos de mensagens e hashes seguros

Referência:
https://docs.python.org/pt-br/3.15/library/hashlib.html#

Este módulo implementa uma interface comum para diversos algoritmos de hash. Estão incluídos os algoritmos de hash seguros FIPS SHA224, SHA256, SHA384 e SHA512 (definidos no padrão FIPS 180-4), a série SHA-3 (definida no padrão FIPS 202), bem como os algoritmos legados SHA1 (anteriormente parte do FIPS) e o algoritmo MD5 (definido em internet **RFC 1321**).

Algoritmos de hash

Há um método construtor nomeado para cada tipo de *hash*. Todos retornam um objeto hash com a mesma interface simples. Por exemplo: use sha256() para criar um objeto hash SHA-256. Agora você pode alimentar este objeto com objetos bytes ou similares (normalmente bytes) usando o método update. A qualquer momento, você pode solicitar o *digest* da concatenação dos dados alimentados até o momento usando os métodos digest() ou hexdigest().
Para permitir multithreading, a GIL do Python é liberada ao calcular um hash fornecido com mais de 2047 bytes de dados de uma só vez em seu construtor ou método .update.

Uso

Para obter o resumo da string de bytes b"Nobody inspects the spammish repetition":

```
import hashlib
m = hashlib.sha256()
m.update(b"Nobody inspects")
m.update(b" the spammish repetition")
m.digest()
b'\x03\x1e\xdd}Ae\x15\x93\xc5\xfe\\\x00o\xa5u+7\xfd\xdf\xf7\xbcN\x84:\xa6\xaf\x0c\x95\x0fK\x94\x06'
m.hexdigest()
'031edd7d41651593c5fe5c006fa5752b37fddff7bc4e843aa6af0c950f4b9406'
```

Mais condensado:

```
hashlib.sha256(b"Nobody inspects the spammish repetition").hexdigest()
'031edd7d41651593c5fe5c006fa5752b37fddff7bc4e843aa6af0c950f4b9406'
```

Atributos

O hashlib fornece os seguintes atributos de módulo constantes:

hashlib.**algorithms_guaranteed**

Um conjunto contendo os nomes dos algoritmos de hash com suporte garantido por este módulo em todas as plataformas. Observe que “md5” está nesta lista, apesar de alguns fornecedores originais oferecerem uma estranha construção Python “compatível com FIPS” que o exclui.

Adicionado na versão 3.2.

hashlib.**algorithms_available**

Um conjunto contendo os nomes dos algoritmos de hash disponíveis no interpretador Python em execução. Esses nomes serão reconhecidos quando passados para new(). algorithms_guaranteed sempre será um subconjunto. O mesmo algoritmo pode aparecer várias vezes neste conjunto com nomes diferentes (graças ao OpenSSL).

Adicionado na versão 3.2.

Objetos hash

Os seguintes valores são fornecidos como atributos constantes dos objetos hash retornados pelos construtores:

hash.**digest_size**

O tamanho do hash resultante em bytes.

hash.**block_size**

O tamanho do bloco interno do algoritmo de hash em bytes.

Um objeto hash tem os seguintes atributos:

hash.**name**

O nome canônico deste hash, sempre em minúsculas e sempre adequado como parâmetro para new() para criar outro hash deste tipo.

Alterado na versão 3.4: O atributo name está presente no CPython desde o seu início, mas até o Python 3.4 não era especificado formalmente, então pode não existir em algumas plataformas.

Um objeto hash tem os seguintes métodos:

hash.**update**(*data*)

Atualiza o objeto hash com o objeto bytes ou similar. Chamadas repetidas são equivalentes a uma única chamada com a concatenação de todos os argumentos: `m.update(a); m.update(b)` é equivalente a `m.update(a+b)`.

hash.**digest**()

Retorna o resumo dos dados passados ao método update() até o momento. Este é um objeto bytes de tamanho digest_size que pode conter bytes em todo o intervalo de 0 a 255.

hash.**hexdigest**()

Similar a digest(), exceto que o resumo é retornado como um objeto string de comprimento duplo, contendo apenas dígitos hexadecimais. Isso pode ser usado para trocar o valor com segurança em e-mails ou outros ambientes não binários.

hash.**copy**()

Retorna uma cópia (“clone”) do objeto hash. Isso pode ser usado para calcular com eficiência os resumos de dados que compartilham uma substring inicial comum.